《算法分析与设计》

实验报告

学　　号　20241126339

姓　　名　毛露涵

年　　级　2024 级

专　　业　软件工程

院　　系　计算机与人工智能学院

实验预习（40%）	上机实验（40%）	文档排版（20%）	总分	批阅人签字

2025-2026 学年第 2 学期

实验 5.3 有障碍物的不同路径数量

1. 实验目的

（1）理解动态规划算法的求解过程。

（2）分析动态规划算法的时间复杂度，比较动态规划算法与其他算法的效率差异。

（3）学会如何利用动态规划算法求解具体问题，了解动态规划算法的局限性。

2. 实验任务

（1）完成实验 5.3（有障碍物的不同路径数量）的各项要求。

（2）用 C++语言实现该算法并进行测试。

（3）撰写实验报告，实验报告内容包括实验目的、实验任务、实验环境、实验步骤、实验结果和实验总结等部分。

3. 实验环境

3.1 硬件环境

（1） 计算机：Legion R9000P ADR10H

（2） CPU: AMD Ryzen 9 8945HX with Radeon Graphics

（3） RAM：32GB

3.2 软件环境

（1） 操作系统：Windows11 家庭中文版

（2） 开发工具：Visual Studio 2026

4. 实验步骤及结果

4.1 实验预习

4.1.1 从左下角以最短距离到达 B 点的路径数为多少？

设到达 A1 点的路径数为 X1，到达 A2 点的路径数为 X2。

由于从左下角到 B 点的最短路径只能从 A1 或 A2 转移过来，因此：

到达 B 点的路径数 = X1 + X2

根据图中路径数量可得：

A1 = 4，A2 = 6

所以：

到达 B 点的路径数 = 4 + 6 = 10

4.1.2 从左下角以最短距离到达 D 点的路径数为多少？

设到达 C1 点的路径数为 Y1，到达 C2 点的路径数为 Y2。

由于从左下角到 D 点的最短路径只能从 C1 或 C2 转移过来，因此：

到达 D 点的路径数 = Y1 + Y2

根据图中路径数量可得：

C1 = 6，C2 = 35

所以：

到达 D 点的路径数 = 6 + 35 = 41

4.1.3 写出采用动态规划法求解该问题的递推方程和边界条件（简要描述 dp 数组、数组下标和递推方程的含义）

设 dp[i][j] 表示从左下角出发，以最短距离到达道路交点 (i, j) 的路径数量。

由于最短路径中只能向上或向右走，所以：

dp[i][j] = dp[i-1][j] + dp[i][j-1]

但如果从 (i-1,j) 到 (i,j) 的竖向道路被大厦内部占用，则不能从下方转移。

如果从 (i,j-1) 到 (i,j) 的横向道路被大厦内部占用，则不能从左方转移。

因此实际递推为：

dp[i][j] = 0

如果 i > 0 且竖向道路没有被封闭：

```
dp[i][j] += dp[i-1][j]
```

如果 j > 0 且横向道路没有被封闭：

```
dp[i][j] += dp[i][j-1]
```

边界条件：

dp[0][0] = 1

其中 (0,0) 表示城市左下角，(m,n) 表示城市右上角。

4.1.4 从左下角以最短路径到达 C1 和 C2 点的路径数是多少？

对于样例输入：

5 6

1

2 4 3 2

0 0

动态规划计算过程中：

A1 = 4

A2 = 6

B = 10

C1 = 6

C2 = 35

D = 41

所以：

到达 C1 点的路径数为 6

到达 C2 点的路径数为 35

4.1.5 用 C/C++语言实现该算法的源代码

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    int m, n;
    while (cin >> m >> n) {
        if (m == 0 && n == 0) {
            break;
        }

        int B;
        cin >> B;
        vector<vector<int>> banV(m, vector<int>(n + 1, 0));
        vector<vector<int>> banH(m + 1, vector<int>(n, 0));
        for (int k = 0; k < B; k++) {
            int x, y, z, b;
            cin >> x >> y >> z >> b;
            int r0 = x - 1;
            int c0 = y - 1;
            int r1 = r0 + z;
            int c1 = c0 + b;
            for (int i = r0; i < r1; i++) {
                for (int j = c0 + 1; j < c1; j++) {
                    banV[i][j] = 1;
                }
            }
```

```
            for (int i = r0 + 1; i < r1; i++) {
                for (int j = c0; j < c1; j++) {
                    banH[i][j] = 1;
                }
            }
        }
        vector<vector<long long>> dp(m + 1, vector<long long>(n + 1, 0));
        dp[0][0] = 1;
        for (int i = 0; i <= m; i++) {
            for (int j = 0; j <= n; j++) {
                if (i == 0 && j == 0) {
                    continue;
                }
                if (i > 0 && banV[i - 1][j] == 0) {
                    dp[i][j] += dp[i - 1][j];
                }
                if (j > 0 && banH[i][j - 1] == 0) {
                    dp[i][j] += dp[i][j - 1];
                }
            }
        }
        cout << dp[m][n] << endl;
    }
    return 0;
}
```

4.1.6 分析上述算法的时间复杂度，给出复杂度结果

设城市大小为 m * n，大厦数量为 B。

程序需要先根据输入的大厦信息标记被大厦内部占用的道路。若第 k 座大厦占用的范围为 z_k * b_k，则标记该大厦相关道路的时间复杂度为：

O(z_k * b_k)

所有大厦的道路标记总时间复杂度为：

O(Σ(z_k * b_k))

之后动态规划需要遍历所有道路交点，交点数量为：

(m + 1) * (n + 1)

因此动态规划部分的时间复杂度为：

O(mn)

所以算法总时间复杂度为：

O(mn + Σ(z_k * b_k))

4.2 上机实验

4.2.1 上机调试，利用实验教程上的测试用例验证程序是否正确

输入：

5 6

1

2 4 3 2

0 0

输出：



程序输出结果：192

实验教程样例结果为 192，程序输出结果与样例结果一致

4.2.2 设计新的测试用例，对程序进行进一步验证

测试样例：

2 3

1

1 1 2 2

0 0

手工计算结果：4

程序输出结果：



程序输出结果：4
程序输出结果与手工计算结果一致。

5. 实验总结

本次实验使用动态规划算法求解有大厦阻隔情况下的不同最短路径数量问题。实验的关键是正确理解题意：大厦占用多个小房子后，大厦内部原有街道不能通行，但大厦边界道路仍然可以通行。

在算法设计中，使用 dp[i][j] 表示从左下角到达道路交点 (i,j) 的最短路径数量。每个位置的路径数由下方和左方两个位置转移得到。如果对应道路被大厦内部占用，则该方向不能转移。

通过样例测试，程序输出结果为 192，与题目给出的结果一致。通过额外设计的小规模测试用例，手工计算结果为 4，程序输出也为 4，进一步验证了算法的正确性。

本算法时间复杂度为 O(mn + Σ(z_k * b_k))，空间复杂度为 O(mn)，适合求解本实验规模下的路径计数问题。